

LAB2. DISTRIBUTED PROTEIN MATCHING in PYTHON

Creation of the datasets

In the lab material you would find a file named “proteins-generator.py”. You have to use to generate proteins datasets for the lab. To generate a data set, execute the command:

To generate a data set, execute the command:

```
$>python proteins-generator.py numrows seed
```

Being “numrows” a parameter specifying the number of protein chains in the dataset, and “seed” is an integer number that will be the seed for random generation.

For development:

```
$>python proteins-generator.py 50000 <seed>
```

To test performance of the solution to deliver a least 2,000,000 rows, but you can test as many as you want:

```
$>python proteins-generator.py 5000000 <seed>
```

The file “proteins.csv” created includes a data set with a list of protein chains, including the following information per protein:

```
"protid","enzyme","hydrofob","sequence"
```

The sequence is the description of the protein components expressed as a chain of chars, where you have to look for matches.

IMPORTANT: Do not modify, touch the file or create transformed fields. For the lab delivery extra files will not be accepted. We will use the same command to generate the dataset.

Laboratory Description

You are asked to create a program to match a pattern introduced using the keyboard against all the proteins in the dataset using Python.

Part zero

Create a dataset "proteins.csv" with at least 5,000,000 lines to be used for final tests in the lab.

Note: for development we advise you to create something smaller (e.g. 50,000 lines).

Part one

Make a serial program in Python that:

1. Reads the pattern to search from the keyboard.
2. Changes the pattern to UPPERCASE
3. Start execution time
4. Reads the protein patterns from the file in pairs (id, sequence)
5. Look for occurrences of the pattern string inside each protein sequence
6. If there are occurrences, register the id of the protein and the number of occurrences in the sequence
7. Show the execution time
8. Print a barchar of occurrences using protein id as X and number of occurrences as Y, using matplotlib.pyplot. Represent the 10 proteins with more matches. If several exist with same number of matches, use them ordered by the maximum Hydrofob.
9. Print in the screen the id of the protein with max occurrences. If several exist with same number, print the one with max. hydrofob.

Part two

- 10.- Write a parallel version of your program using **MPI**. Optimize the program to get the fastest version you can.
- 11.- Repeat execution, measuring the time before printing the histogram and the protein with max occurrences.
- 12.- Show the speedup compared with the serial version.

Part three

- 13.- Write a report explaining your results (maximum 12 pages). Explain which technique is better. Show charts with performance comparison.

Laboratory Delivery

Groups of 4 students.

You have to deliver a single compressed file per group. The file will be named:

"yournia_lab2_2026.zip" (e.g. 100052132_lab2_2026" including:

- Report with the memory in PDF (include author names)
- Authors file. One line per author.
- Python program with the serial version of the program.
 - Name: "serial-proteins.py".
- Python program with the parallel version of the program with MPI.
 - Name: "mpi-proteins.py".

Delivery date: October 13th 2026. 23:30 hours.

DO NOT deliver Jupyter notebooks. Just Python scripts.